

' TYPES OF SEARCH '

Path Search: Optimal sequence of actions to reach to the goal.

Key consideration is the path cost.

Like A*, UCS, BFS, DFS

Configuration Search: Best set of parameters that maximizes objective function

The goal itself is important, not the path.

Like Hill climbing, genetic algorithm, CSP

CONSTRAINT SATISFACTION PROBLEM (CSP)

Solving a problem under certain constraints or rules.

A constraint satisfaction problem consists of three components

- $X \rightarrow$ a set of variables, $\{X_1, \dots, X_n\}$
- $D \rightarrow$ a set of domain, $\{D_1, \dots, D_n\}$
- $C \rightarrow$ a set of constraints specifying allowable combinations of values for subset of variable X_i

Constraint values consist of pair of (scope, rel)

scope: variables that participate, rel: defines values of variables can take on

e.g. If X_1 and X_2 both have domain $\{1, 2, 3\}$, then constraint saying that X_1 must be greater than X_2 can be written as

$$\langle (X_1, X_2), \{(3, 1), (3, 2), (2, 1)\} \rangle \text{ or } \langle (X_1, X_2), X_1 > X_2 \rangle$$

Example Problem : Map Coloring ...

$$X = \{WA, NT, Q, NSW, V, SA, T\}$$

$$D = \{red, green, blue\}$$

$$C = \{SA \neq WA, SA \neq NT, SA \neq Q, SA \neq NSW, \\ SA \neq V, WA \neq NT, NT \neq Q, Q \neq NSW, \\ NSW \neq V\} \text{ "Two neighboring regions can't have same color"}$$

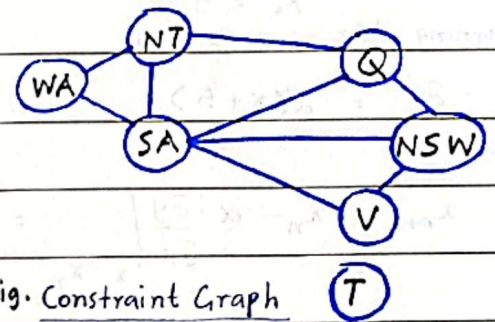


fig. Constraint Graph

Here $SA \neq WA$ is a short cut for

$$\langle (SA, WA), SA \neq WA \rangle \text{ or } \langle (SA, WA), \{(red, green), (red, blue), (green, red), (green, blue), \\ (blue, red), (blue, green)\} \rangle$$

Possible Solution: $\{WA = red, NT = green, Q = red, NSW = green, V = red, SA = blue, T = green\}$

Goal of CSP is to assign values to variables such that all constraint are satisfied.

It can quickly prune large swathes of search space.

Date: 6-3-26

2.5-2 ~ AI -

A search procedure that doesn't use constraints would have to consider $3^5 = 243$ possible assignments, with constraints only $2^5 = 32$ assignments a reduction of 87%.

Job - shop scheduling . . .

$X = \{ \text{Axle F, Axle B, Wheel RF, Wheel LF, Wheel RB, Wheel LB, Nuts RF, Nuts LF, Nuts RB, Nuts LB, Cap RF, Cap LF, Cap RB, Cap LB, Inspect} \}$

$D = \{ 0, 1, 2, \dots, 30 \}$ "The whole job must finish within 30 min."

$C \rightarrow$ 1. Precedence constraints, 2. Disjunctive constraints, 3. Final Inspection constraint

1- If task T1 must happen before T2 and T1 takes d minutes then:

$T1 + d \leq T2$ (means T2 can start only after T1 finishes)

e.g. Axle must be installed before wheels, if d = 10 min then:

$\text{Axle F} + 10 \leq \text{Wheel RF}$

$\text{Axle B} + 10 \leq \text{Wheel RB}$

$\text{Axle F} + 10 \leq \text{Wheel LF}$

$\text{Axle B} + 10 \leq \text{Wheel LB}$

(Front Axle must be installed before front wheels) ----- same for Back -----

2- (Shared tool) there is only one tool for installing axles. So, Axle F and Axle B can't happen at same time.

$\text{Axle F} + 10 \leq \text{Axle B}$ or $\text{Axle B} + 10 \leq \text{Axle F}$

3- Inspection must happen after all tasks finish. It takes 3 minutes for every task X:

$X + d_X \leq \text{Inspect}$

e.g. $\text{Cap RF} + 1 \leq \text{Inspect}$

$\text{Cap LF} + 1 \leq \text{Inspect}$

The CSP solver will:

Assign start time to each task.

Ensure tasks happen in correct order.

Prevent tool conflict.

Ensure all work finishes before inspection.

Finish entire assembly within 30 minutes.

DUA

Date: 6-3-26

.. AI ..

N - Queens ...

$$X = \{ Q_1, Q_2, \dots, Q_n \}$$

$$D = \{ 1, 2, \dots, N \}$$

$$C = \{ \forall i, j \text{ non-threatening } (Q_i, Q_j) \} \text{ i.e. queens must not attack each other.}$$

.	Q	.	.
.	.	.	Q
Q	.	.	.
.	.	Q	.

One valid assignment: $(Q_1, 2), (Q_2, 4), (Q_3, 1), (Q_4, 3)$

Sudoku ...

$$X = \{ \text{cells } (A1 - I9) \} \quad \begin{array}{l} \text{Rows } 1 - 9 \\ \text{Columns } 1 - 9 \end{array} \quad \text{Total 81 variables}$$

$$D = \{ 1, 2, 3, \dots, 9 \} \text{ values each cell can take}$$

$$C = \{ 9 \text{ possibilities all diff for each row} \}$$

$$= = = = = = = = = \text{column}$$

$$= = = = = = = = = \text{region (3x3 grid)}$$

Some cells are already filled in the puzzle, act as "unary constraints".

: Variation on the CSP formalism :

The types of CSP depends mainly on: Types of domains of variables.

Types of constraints between variables.

"Types of domain in CSP"

The simplest type of CSP occurs when variables have **discrete** values and domain is **finite**.

e.g. Map Coloring problem, Scheduling with time limits, 8-Queens Problem.

Some CSPs have **discrete** domains that are **infinite**.

e.g. set of integers or strings, job scheduling without deadline (can start at any time so infinite possible values)

Note: With infinite domains, we cannot list all possible combinations instead use

implicit constraints. like $T1 + d1 \leq T2$

Linear Constraints: Variables in only linear form (only 1 power).

e.g. $T1 + d1 \leq T2$, $2X + 3Y \leq 10$, $X + Y = 5$ etc.

If CSP constraints are linear, special algorithms can solve them efficiently.

Example areas, scheduling, resource allocation, planning

Non linear Constraints: like $x^2 + y \leq 10$, $xy = 20$, $\sin(x) + y = 5$

No algorithm exists for solving general non linear constraints.

Date: 6-3-22

22-1-AI-1

CSPs with **continuous domains** are common in real world.

Continuous variables can be any real numbers within a range. (time = 2.453 sec)
(distance = 4.873 km)

e.g. Hubble Space Telescope Scheduling:

Variables: start time of observation.

End time of observation.

Spacecraft maneuver times.

One of the most famous continuous CSP categories is **Linear Programming**.

Here variables are continuous and constraints must be linear equalities or inequalities.

Linear programming problems can be solved in polynomial time. This means they are **efficiently solvable**.

Other Related Optimization Problems

Researchers also study problems with **different constraints and objective functions**:

- **Quadratic Programming**: Constraints linear, Objective function quadratic.

- **Second-Order Conic Programming**:

"Types of Constraints in CSP"

Unary Constraint restricts value to single variable.

e.g. In map coloring suppose South Australia can't be colored green.

So, constraint is $\langle SA, SA \neq \text{green} \rangle$

NOTE: Even defining the domain of a variable can be considered a unary constraint.

Binary Constraint relates to two variables. A binary CSP can be represented

e.g. $SA \neq NSW$

using constraint graph.

Higher-order constraints:

Ternary Constraint, a constraint involving 3 variables.

e.g. Between (X, Y, Z) , $X < Y < Z$ or $X > Y > Z$ (meaning Y must lie between X & Z)

Global Constraint involves an arbitrary number of variables.

Note: "global" doesn't mean all variables. It means many variables maybe involved.

e.g. **AllDiff** constraint - all variables that are involved must have different values

Date: 6-3-26

25. - AI -

Example: CRYPTARITHMETIC

In cryptarithmic puzzle, each letter stands for distinct digit, (from 0 to 9)

No two letter have same value,

After substitution resulting sum must be correct

Example :-

Variables: $X = \{T, O, G, U\}$

$O = 1$

T O

Domain: $D = \{0, 1, \dots, 9\}$

$T = 2$

G O

Constraint: $C = \text{Alldiff}(T, O, G, U)$

$G = 8$

O U T

$U = 0$

→ This is supposed to be left most digit which is resultant of addition of (T, G)'s carry.

So, it has to be 1.

T O	2 1
G O	8 1
O U T	1 0 2

$$O + O = 1 + 1 = 2 = T$$

$$T + G = 2 + 8 = 10 \quad (\text{This must generate Carry} = 1)$$

try #1. $2 + 9 = 1$; Carry = 0 = 1 (Invalid b/c $O=1$ and $U=1$ constraint violation)

try #2. $2 + 8 = 0$; $O = 1$ (valid $\text{Alldiff}(T, O, G, U)$)

Note: Solution to cryptarithmic puzzle is not unique

like $\begin{array}{r} \text{TWO} \\ \text{TWO} \\ \hline \text{FOUR} \end{array}$ has $\begin{array}{r} 765 \\ 765 \\ \hline 1530 \end{array}$, $\begin{array}{r} 836 \\ 836 \\ \hline 1672 \end{array}$, $\begin{array}{r} 867 \\ 867 \\ \hline 1734 \end{array}$ etc solutions.

Constraint Hypergraph

Date: 7-3-26

Polynomial Time (PT) → can be solved using Deterministic Turing Machine

An algorithm runs in polynomial time if its running time can be written as

$$T(n) = O(n^k) \text{ where } n = \text{input size, } k = \text{constant}$$

This means running time grows reasonably with input size.

e.g. Running Time $T(n) = 3$ → constant time

$T(n) = n$ → Linear time

$T(n) = n \log n$ → Slightly faster than quadratic

$T(n) = n^3$ → Still polynomial

Non Polynomial Time (NP) → Solved by Non-Deterministic Turing Machine
Turn to approximation algos.

Running Time $T(n) = 5^n$ } Non polynomial

$T(n) = n!$ } Exponential time algorithm

They become impossible to run for large inputs

$$n=20, 20! = 2432902,008,176,640,000$$

P problems: that can be solved in polynomial time e.g. sorting, shortest path, MST

NP problems: finding solution in PT is hard, but checking solution is easy.

e.g. Sudoku → finding correct grid is hard (9^{81} possibilities)

→ verification of solution is fast ($TC = O(n^2)$)

NP-complete problems: To solve such problems either use powerful hardware (supercomputers, parallel computing) or wait for long time.

An other most common approach is to use approximation algorithms.

Approximation Ratio -

$$\rho(n) = 1$$

solution is perfect

$$\rho(n) \geq \max \left(\frac{C}{C^*}, \frac{C^*}{C} \right)$$

C^* - cost of optimal sol.

C - cost produced by approximation algo.

e.g. $C^* = 20$ → optimal cost of MST

$C = 25$ → tree's cost from approx. algo

$$\rho(n) = \frac{C}{C^*} = \frac{25}{20} = 1.25$$

(The algorithm solution is almost 1.25 times the optimal one.)

Date: 7-3-26

~AI~

"Real World CSPs"

Many CSPs can't be solved by inference alone. So, we must "search" for the solution.

Search Formulation for CSPs (Incremental Formulation)

States - Current assignments made so far.

Initial state - The empty assignment $\{\}$

Successor Function - Assign a value to an unassigned variable that doesn't conflict with constraints

Goal Test - The current assignment is complete and satisfies all constraints.

Path Cost - Constant cost for every step (not really relevant)

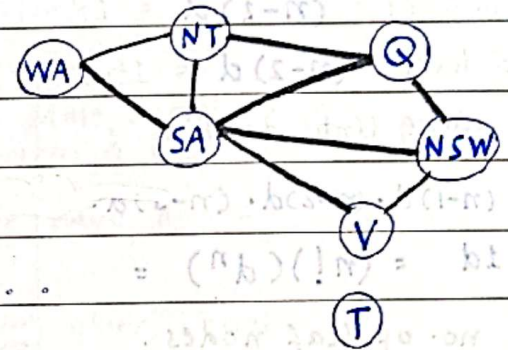
- BFS -

Initial State: Empty Assignment $\{\}$

Successor Function: Starting from WA

(Partial Assignments) $\leftarrow \{WA, R\}, \{WA, G\}, \{WA, B\}$

Similarly for NT
 $\{NT, R\}, \{NT, G\}, \{NT, B\} \dots$



Expanding variables:

Two variables - $\{WA=R, NT=R\}, \{WA=R, NT=B\}, \{WA=R, NT=G\}$
 $\{WA=B, NT=R\}, \{WA=B, NT=B\}, \{WA=B, NT=G\} \dots$

Three variables - $\{WA=R, NT=B, SA=G\}, \{WA=R, NT=B, SA=R\}, \dots$

! Drawback :: BFS must explore search tree entirely level by level.

Therefore, It must generate a huge number of partial assignments.

If Variables = 7, then possible assignments $3^7 = 2187$ node maybe explored by BFS.
Domain = 3

- DFS -

DFS works differently instead of exploring level by level it goes deep into tree.

Initial: $\{\}$

"Assigning variables $\{WA=R\}$

$\{WA=R, NT=B\}$

until $\{WA=R, NT=B, SA=G\}$

complete" $\{WA=R, NT=B, SA=G, Q=R, NSW=B, V=R, T=G\}$

This complete assignment. No algo check "Do all constraints hold?"

If yes $\rightarrow$ SOLUTION FOUND

If no $\rightarrow$ BACKTRACK

Date: 7-3-26

- AI -

" Backtracking Search for CSPs "

Backtracking search is the basic uninformed algorithm for solving CSPs

Basic Idea :-

Assign a value to a variable

check if constraints are satisfied

If valid $\rightarrow$ continue

If invalid $\rightarrow$ undo the assignment (backtrack) and try another value.

It work like DFS . Backtracking = DFS + variable-ordering + fail-on-violation

For CSP with n variables

of domain size d .

Branching factor at top level

$$= nd = 7 \times 3 = 21$$

$$\text{Second level} = (n-1)d = 18$$

$$\text{Third level} = (n-2)d = 15$$

Continuing Until depth n :

$$nd \cdot (n-1)d \cdot (n-2)d \cdot (n-3)d \cdot$$

$$\dots 1d = (n!)(d^n) =$$

Total no. of leaf nodes .

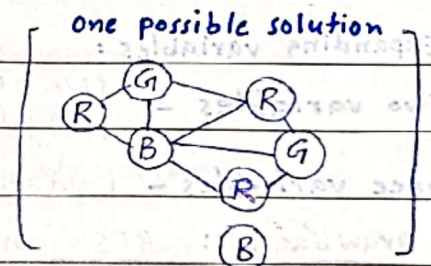
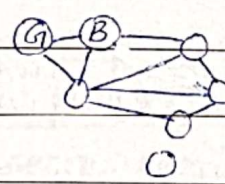
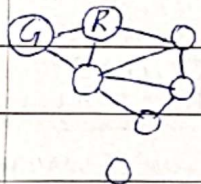
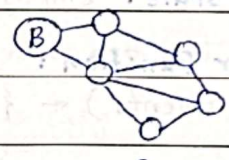
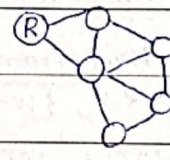
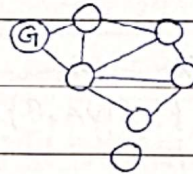
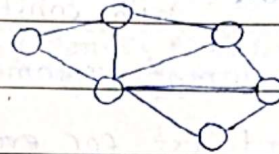
e.g. For australia map

$$\text{Total leaf nodes} = 7! (3^7)$$

$$= 11,022,480$$

That is, 11 million leaves .

7 variables
3 domain



! why is the tree so LARGE ? Because **NAIVE SEARCH** repeats same assignments in different orders e.g. Path 1 WA=R, NT=G } Both produce { WA=R, NT=G }
Path 2 NT=G, WA=R } But naive search explores both

COMMUTATIVITY PROPERTY :

Instead of choosing any variable at any time we :

- Choose one variable at each level
- Assigns value only to that variable

Then **Branching Factor** becomes d^n

$n!$ was just extra work because CSPs are commutative so, those all lead to same final assignment as discussed above.

Date: 7-3-26

25-2 -- AI --

Improving Backtracking

General purpose ideas that give huge gains in speed

[Which variable should be assigned next? → ORDERING
and In what order should its values be tried?]

[Can we detect inevitable failure early? → FILTERING]

[Can we exploit the problem structure? → STRUCTURE]

"variable & value ORDERING"

→ Minimum - Remaining - Values (MRV) (Most constrained variable or Fail-first Heuristic)

MRV heuristic selects : variable that has the smallest no. of legal values remaining.

Focuses on most restricted variables first (one with fewest options left)

Hence, detects conflicts immediately.

Prevents exploring useless branches.

Tie Situation :- Sometimes two variables have same domain size.

For example initially every region has three legal colors, this is called Tie.

For that we have another heuristic

→ Degree - Heuristic

Choose variable involved in most constraints with other variables

e.g. like SA is connected with 5 regions. So choose SA first.

It attempts to reduce branching factor on future choices.

It can be useful as tie-breaker.

"Once variable is selected, the algo. must decide on order in which to examine its values - - - - -"

→ Least-Constraining-Value - Heuristic

Choose value that eliminates fewest options for neighbors.

e.g. suppose $WA = R$, $NT = G$,

Now selecting value for Q → R Choose RED.

→ B choosing blue removes all choices for SA (Bad)

It leaves maximum flexibility for future variables.

Variable ... Choose most constrained → detects failure quickly.

Values ... Choose least constrained → keeps more possibilities.

"FILTERING"

Keep track of domains for unassigned variables and cross off bad options.

→ Forward Checking

After assigning a variable, remove inconsistent values from neighboring variables.

e.g. If $WA = R$, then $NT \neq R$, $SA \neq R$. It improves efficiency & detects failure early.

Date: 7-3-26

AI -

But if $WA = R$ and $Q = G$, then NT and SA both require Blue though they are neighbors so, "conflict occurs."

75

limitation: Forward Checking only checks Direct Neighbors.
It doesn't propagate further constraints, which cause inconsistency.

→ Maintaining Arc Consistency (MAC)

A stronger inference method. Uses AC-3 algorithm during search.

Arc Consistency - Two variables X and Y 's constraint is arc consistent if there exists at least one value of X that satisfies or is compatible with Y 's value (domain).

e.g. $X \neq Y$ if $X = \{R, G\}$ $Y = \{R\}$ this violates constraint so, remove it.

Final Domains $X = \{G\}$, $Y = \{R\}$. Now the arc (X, Y) is consistent.

MAC add all arcs (X_j, X_i) to a queue,

And run AC-3 to enforce arc consistency (to remove inconsistent values from domains)

If any domain becomes empty, we immediately Backtrack.

Remember: AC alone can't find solution. In most of the cases backtracking is needed.

It is used to prune domain in CSPs while it helps to reduce search space, it doesn't guarantee a solution.

After Arc Consistency:

a) One solution Left: When only one value is remaining for each variable that satisfies all neighbouring constraints.

At this point the solution is essentially determined.

b) Multiple solutions left: Some variables may still have more than one value in their domain.

AC can't eliminate all ambiguities because some constraints involve combinations of multiple variables.

Here backtracking search is needed to further explore multiple assignments.

c) No solution Left: AC only looks at binary constraints locally. There can be situations where, globally, no combination satisfies all constraints.

This is because AC removes values inconsistent with neighbouring variables.

25. - AI -

Local Search for CSPs

-DUA

Date: 7-3-26

DS - AI -

Local Search for CSPs

"SAATCHI2 0918079"

Local search algorithms turn out to be very effective in solving CSPs.

The problem is represented as complete-state formulation, where each variable is assigned to some value, even if some constraints are violated. We then randomly choose a conflicted variable to change the value to something that brings us closer to a solution.

Min-Conflict Heuristic - the most common local search method for CSPs

"It moves a variable to the value that creates the fewest constraint violation."

★ On the n -queen problem, if you don't count initial placement of Queens, the run-time of MCH is roughly independent of problem size. It solves millions queen problem in an average of 50 steps.

★ MCH has been used to "schedule observations" for Hubble Space Telescope, reducing time taken to schedule a week of observations from 3 weeks to around 10 minutes.

Plateaus in Local Search - Plateaus occur when many states have same conflict values. Moving between them doesn't improve the score. The search wanders without progress.

- 1) To solve this we allow **sideway moves**. Means move to another state with same conflict value. This way we may find a better state.
- 2) Another technique to avoid wandering on plateaus is **tabu search**. It keeps a small list of recently visited state, forbidding the algorithm to return to those states.
- 3) **Simulated Annealing** can also be used to escape from plateaus. That sometimes accepts worst states temporarily to escape local minima.
- 4) **Constraint Weighting** also aims to concentrate the search on important constraints.
→ Each constraint is given weight = 1 initially.
During search if constraint is violated "increase its weight."

Because of these weights flat area becomes sloped surface, so improving moves become visible.

It also helps to learn which constraints are hardest and adds topography to plateaus.

Date: 9-3-26

AI - 1

"Adversarial Search"

Search in competitive environments, where multiagents are searching for one solution in same search space like in Games, is called "Adversarial Search" often known as "game theory".

Most AI game problems are :-

- ♦ **DETERMINISTIC** - No randomness, the next state is completely determined by current state + chosen move. Like tic-tac-toe, chess, Go etc
- ♦ **TWO-PLAYER** - Exactly two opponents. One tries to win and other tries to stop them.
- ♦ **TURN-TAKING** - Players moves one after another, not at the same time.
- ♦ **PERFECT INFO (Fully Observable)** - Both player can see the full environment.
- ♦ **ZERO-SUM** - One player's gain = Other player's loss.
If MAX benefits, MIN is hurt by same amount.
- ♦ **General Sum Games** - Agents have independent utilities ♦ **Team Games** - Common payoff for all team members.

Formal Definition of a Game

Initial State: How game starts e.g. Empty tic-tac-toe board.
Standard chess setup.

To-Move: Tells whose turn it is state s .

Actions: All possible moves available.

Result (Transition Model): What happens if action a is taken in state s .

$RESULT(s, a) \rightarrow$ new state.

Is-Terminal (Terminal Test): Check if game is over. $s \rightarrow \{T, F\}$

True $\rightarrow$ game ended, False $\rightarrow$ game continues

Utility (Payoff Function): Final score for player p if game ends in state s .
Only defined for terminal states.

State Space Graph

All possible states and moves are shown. Graph where vertices are states and edges are moves.

Search Tree

A tree built from the current position to explore future moves.

Game Tree

A search tree that follows every sequence of moves all the way to terminal. Leaves are game outcome.

(!) Game tree can be enormous, so, full game trees are usually theoretical.

Date: 9-3-26

:- AI :-

(MINIMAX Algorithm)

MAX wants to win and MIN wants to stop MAX.

So, MAX can't plan a fixed sequence of moves.

Strategy of MAX must be Conditional:

- Observe every possible response by MIN.
- Specify moves in response

This optimal strategy can be determined from minimax value of each node.

The minimax value in terminal state is utility.

In non-terminal state it is MAX's preferred step.

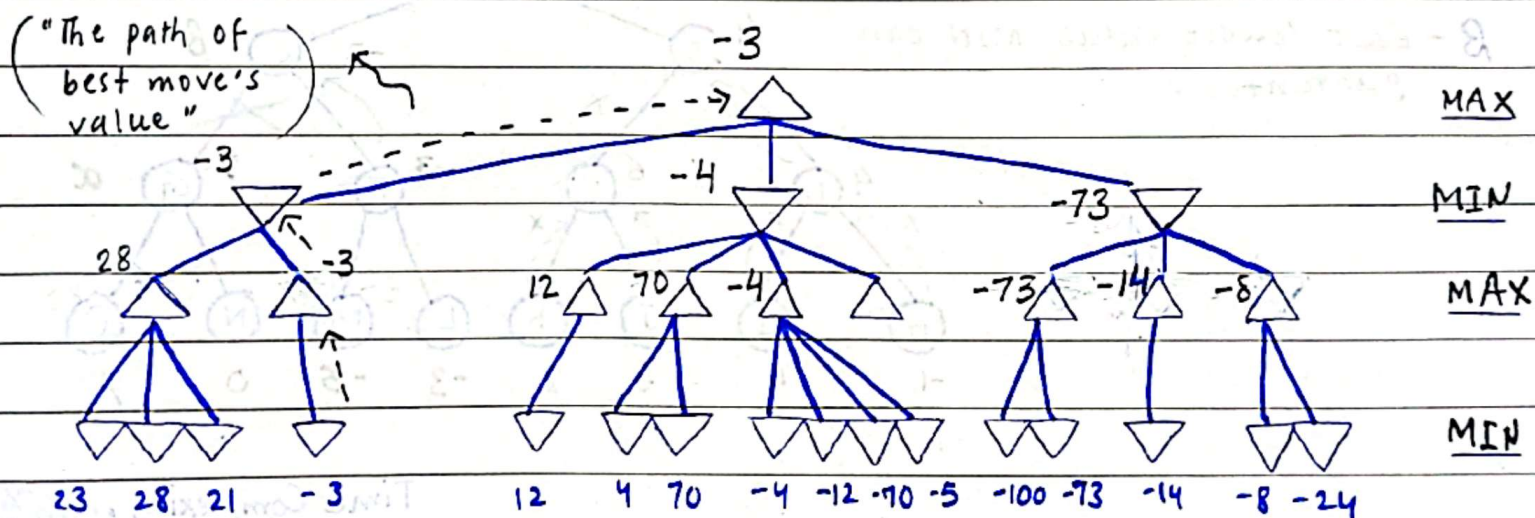
Maximum of its MIN children $\rightarrow$ MAX = 4 (A) $\times$ (4 is minimax value ----)

Minimum of its MAX children $\rightarrow$ MIN = 4 (B) $\times$ (C) = -3

Maximum of its MIN Children $\rightarrow$ MAX = 4 (D) $\times$ (E) = 6 (F) = -3 (G) = 7

Utility Values $\rightarrow$ -1 4 2 6 -3 -5 0 7

Path by which we got value of A :- A $\rightarrow$ B $\rightarrow$ D $\rightarrow$ I



NOTE :-

Utility (S, P): Defines the final numeric value for a game that ends in "terminal state S" for player P.

Minimax (S, P): Defines numeric value at all other nodes.

DUA

Date: 10-3-26

- AI -

" Properties of minimax algorithm "

Time Complexity • $O(b^m)$

Space Complexity • $O(bm)$

Complete: Yes, if tree is finite

Optimal: Yes

" Limitation "

In some cases game tree can be huge that it will become difficult to traverse.

Hence, we can't use "minimax algorithm" in every game.

To overcome this problem we use "alpha-beta pruning" algorithm.

(ALPHA - BETA Pruning)

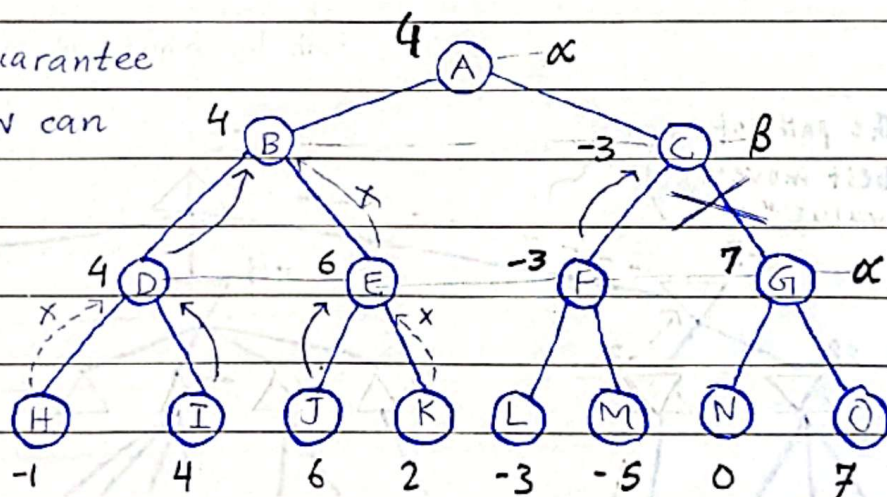
It is possible to compute correct minimax decision without looking at every single node.

Pruning: is ways of determining that certain branches will not be useful.

This is Metareasoning — agent computes "How much search should I do." The AI reasons about its own computation process.

α - Best value MAX can guarantee

β - Best lowest value MIN can guarantee.



Time Complexity • $O(b^{m/2})$